PINKCC

Small talk about
Reproducibility

Chad Estoup--Streiff.

PhD Student at IRCM - PINKCC Lab.

Why having a good level of reproducibility in your code ?

- **Enhances Research Quality:** Open-source code encourages rigorous self-correction and external validation.
- **Boosts Publication Chances:** Many top-tier journals prioritize or require open, reproducible workflows.
- **Increases Citations and Impact:** Making your code accessible allows other researchers to build on your work and cite you.
- **Facilitates Peer Validation:** Open code acts as a stamp of transparency, proving your methodology is sound and reproducible.

Step to make work reproducible

- **Methodology :** good structure, writing and comments: *Clean code*
- **Analysis stack:** gold references for Language, libraries, softwares: *Python 3.15, Numpy, Pytorch*
- **Environment execution:** Cross-plateform and execution consistency: *Docker + UV*

1. Methodology

A. Why it Matters

- **Scientific Transparency:** Code is the "instruction manual" of your math; if it's messy, the manual is unreadable.
- **Reduced Error Rate:** Clean code structure makes bugs easier to spot and harder to hide.
- **Knowledge Transfer:** Ensures that when a PhD student leaves, the lab doesn't lose years of work to "un-runnable" scripts.

1. Methodology

B. Solutions

- **Modularization:** Moving from one 2,000-line main.py to a structured /src folder with dedicated data_processing.py and model_utils.py.

- **Type Hinting:** Using Python’s type system to prevent "Silent Failures" (e.g., passing a String where a Float was expected).

- **Functions documentation:** Each function should be documented with a global description and a precise description of inputs and outputs

- **Auto-Formatters:** Using Black or Ruff to enforce a universal coding style across the entire research team automatically.

Let’s learn and practice with this tutorial

https://testdriven.io/blog/clean-code-python/

2. Analysis stack

A. Why it Matters

- **Community Benchmarking:** Using "Gold Standard" libraries allows other researchers to compare your results against theirs using the same underlying logic.
- **Numerical Stability:** Industry-standard libraries (like NumPy) have been vetted for floating-point errors that custom-written math functions might miss.
- **Future Compatibility:** Popular libraries are maintained for decades, ensuring your code doesn't break six months after publication.

2. Analysis stack

B. Solutions

- **Python 3.15:** Leveraging the latest performance optimizations (like specialized adaptive interpreters).
- **NumPy & PyTorch:** The "Tensor Standard." Using torch.compile for hardware-accelerated research that remains readable.
- **Validation Tools:** Using Pytest to create "unit tests" for your scientific formulas to prove they work before you run the full experiment.
- **Other tools:** just make sure it’s gold standard so other users know how to use it.

3. Environment execution

A. Why it Matters

- **The "Dependency Hell" Cure:** Prevents the "it worked on my laptop" excuse.
- **Bit-for-Bit Reproducibility:** Ensures that X input always equals Y output, regardless of whether it's run on a Mac in 2026 or a Linux server in 2030.
- **Scalability:** Allows you to move your experiment from a local PC to a High-Performance Computing (HPC) cluster instantly.

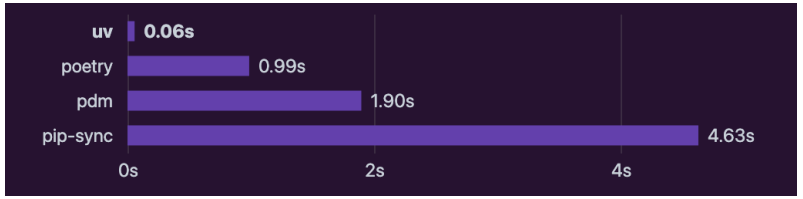
3. Environment execution

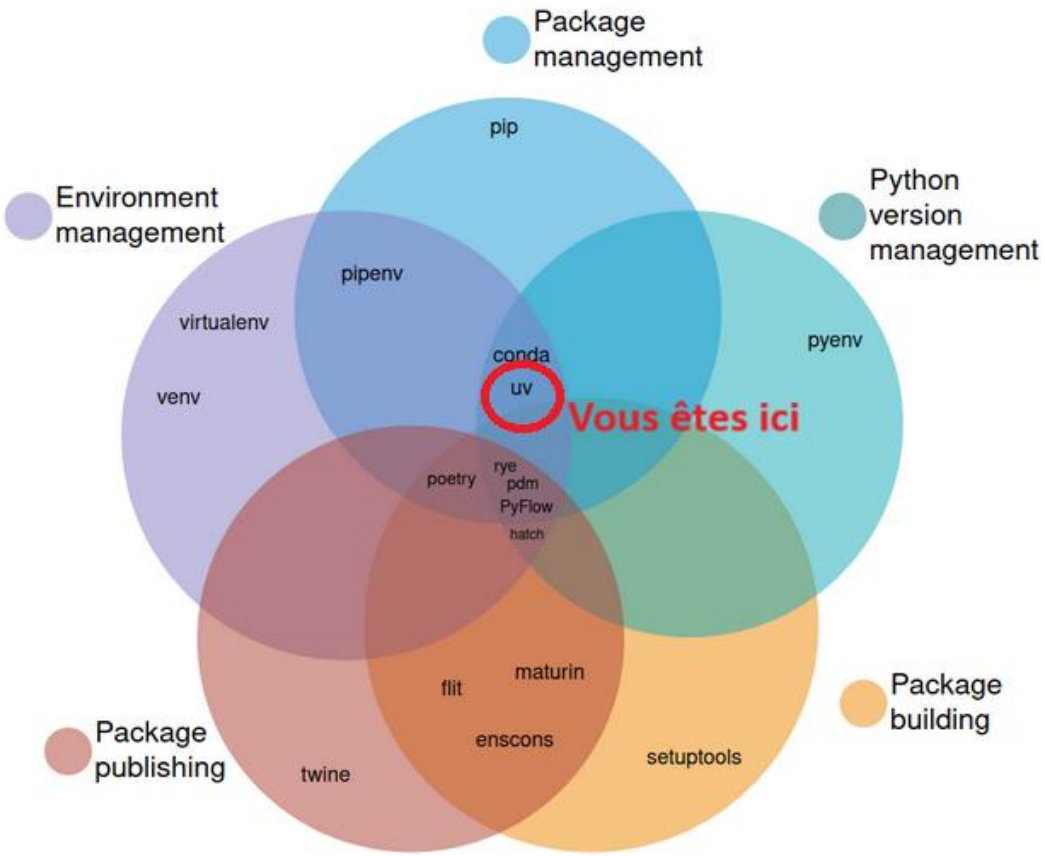
B. Solutions

- **UV (The Modern Standard):** Replacing slow pip installs with a lightning-fast Rust-based resolver. It generates a uv.lock file that freezes your exact environment.
- **Docker:** Wrapping the code, Python version, and system drivers into a Container Image.

What is UV ?

Super fast dependency manager
(thx Rust)



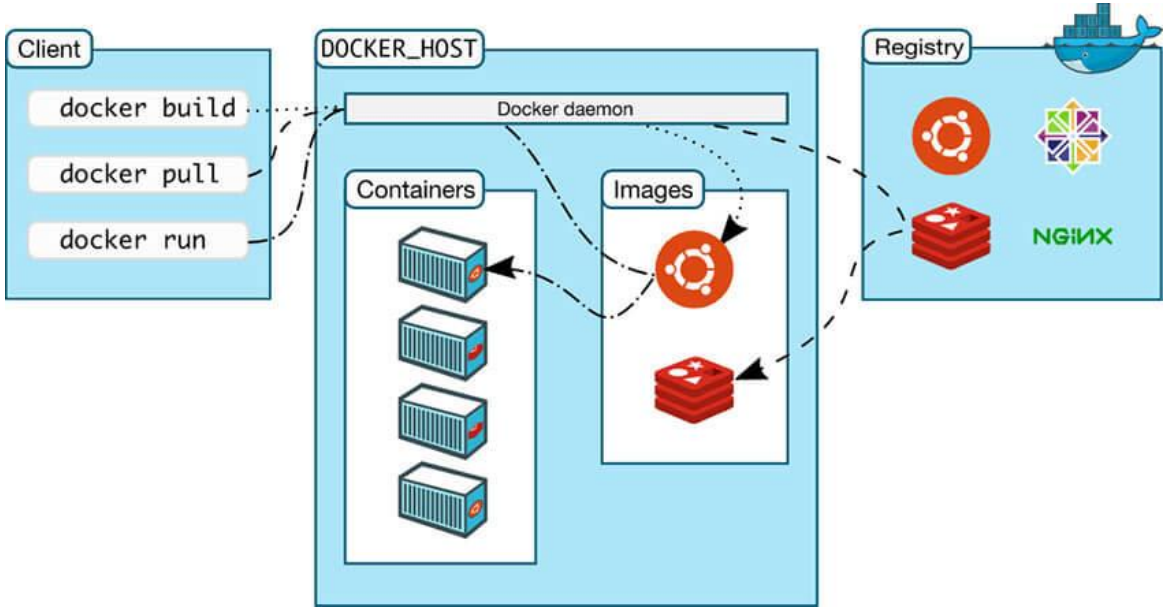


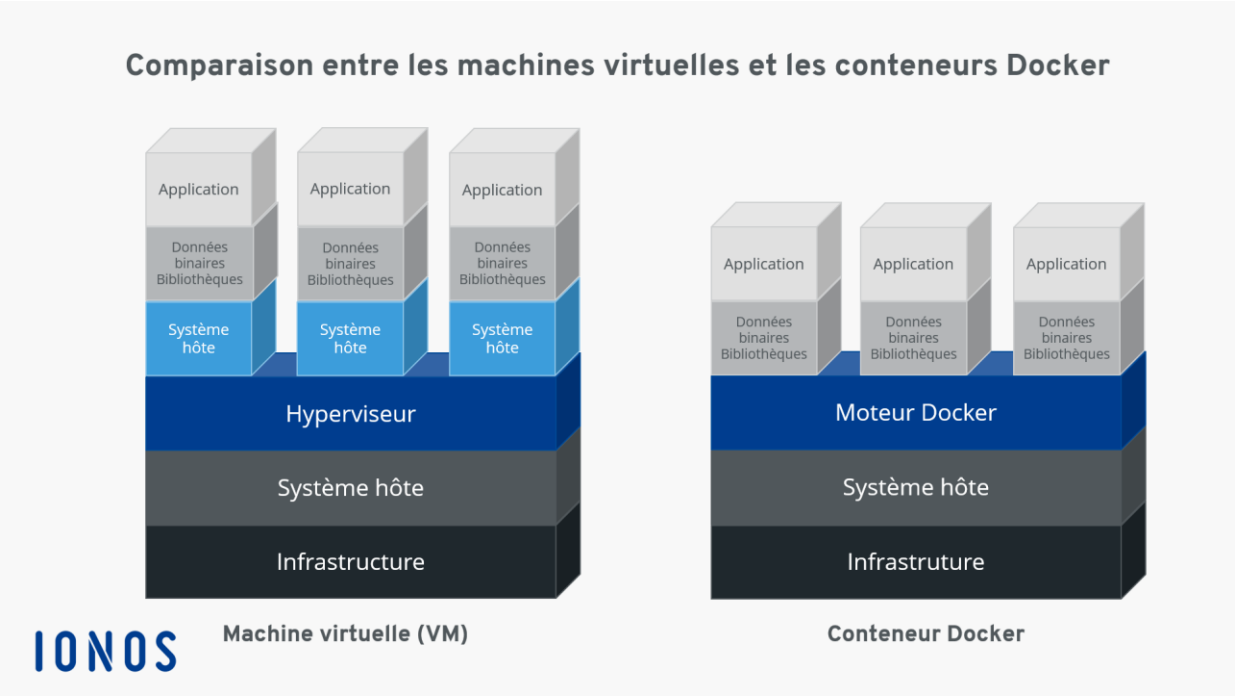
Performance: It is often 10x to 100x faster than pip.

Simplicity: A single tool manages your Python versions, libraries, and virtual environments. Can change python version on the fly.

Reliability: It creates a uv.lock file by default, ensuring your scientific environment is identical every time it’s installed—the "holy grail" of reproducibility.

What is Docker ?





True Reproducibility: It freezes the entire OS environment, ensuring your code behaves exactly the same way in five years as it does today.

Zero Configuration: Collaborators can run your entire research pipeline with one command, without manually installing Python, libraries, or system drivers.

Portability: It eliminates the "it works on my machine" problem, allowing you to move seamlessly from a local laptop to a high-performance computing (HPC) cluster.

The power of the combination of Docker + UV

Feature	uv (The Manager)	Docker (The Container)
Focus	Python libraries and versions	The entire Operating System
Reproducibility	Logical (Lockfiles)	Physical (Images)
Speed	Extremely fast installs	Instant environment launch

The Gold Standard, why this Combo Wins: The "One-Click" Research Pipeline

- **Total Reliability: Docker** freezes the OS, **uv** freezes the libraries. No "it works on my machine" excuses.
- **Future-Proofing:** Your 2026 research stays runnable in 2030. It stops "bit rot" by locking every single dependency in an immutable container.
- **Extreme Speed: uv** removes the "Docker build tax." It installs environments in seconds, making reproducibility a feature, not a chore.
- **Peer Trust:** You move from "Trust me, I did the math" to **"Here is the button to prove it."**

And don't worry, it's very VERY easy 🙇‍♀️

Let’s learn and practice with this tutorial

https://github.com/PINKcc-Lab/docker-uv

Scientific ressources

1. The Reproducibility Crisis — Scope & Scale

- **"Understanding experiments and research practices for reproducibility: an exploratory study"** Samuel & König-Ries, *PeerJ* (2021) — https://pmc.ncbi.nlm.nih.gov/pmc/articles/PMC8067906/ → A survey across disciplines where ~90% of scientists agreed a reproducibility crisis exists. Key finding: insufficient metadata, lack of publicly available data, and incomplete method descriptions are the main causes of poor reproducibility. Excellent broad framing paper. nih
- **"Reproducibility and research integrity: the role of scientists and institutions"** *PMC* (2021) — https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8672590/ → At least 70% of scientists were unable to reproduce studies from other scientists, and at least 50% could not reproduce their own work. Good for a stark opening statistic. nih
- **"Reproducibility, Replicability, and Repeatability: A survey of reproducible research with a focus on high performance computing"** Antunes & Hill, *Computer Science Review* (2024) — https://arxiv.org/pdf/2402.07530 → Examines factors in scientific practices that contribute to the lack of reproducibility across disciplines, with a focus on computational environments and software dependencies. arxiv

Scientific ressources

2. Computational Reproducibility & Environment Management (→ justifies Docker + UV)

- **"An introduction to Docker for reproducible research, with examples from the R environment"** Boettiger, *ACM SIGOPS Operating Systems Review* (2015) — https://arxiv.org/abs/1410.0846 → **The** canonical peer-reviewed paper on Docker for research. Explores common reasons that code cannot be executed by subsequent researchers, reviews virtual machines and workflow systems and their limitations, then examines how Docker combines OS virtualization, cross-platform portability, modular reusable elements, and versioning to address these challenges. ACM Digital Library
- **"Ten simple rules for writing Dockerfiles for reproducible data science"** *PLOS Computational Biology* (2020) — https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1008316 → Argues it should become increasingly straightforward for the average researcher to provide computational environment support for their publication in the form of a Dockerfile or pre-built Docker image. Very practical, citable as best-practices guidance. PLOS
- **"Continuous Analysis: Evolution of Software Engineering and Reproducibility for Science"** (2024) — https://arxiv.org/pdf/2411.02283 → A review of 15,000 bioinformatics tools showed nearly 50% were difficult to install or reproduce due to poorly documented dependencies. In computational materials science, four key issues identified were: lack of reporting computational dependencies, missing version logs, poorly organized code, and unclear references. This directly justifies using UV's lockfile approach. arxiv
- **"More Rigorous Software Engineering Would Improve Reproducibility in Machine Learning Research"** Wolter & Veeramacheneni (2025) — https://arxiv.org/pdf/2502.00902 → Argues that experimental verification of scholarly work requires robust software engineering, and recommends practices including documenting software requirements and providing reproduction steps in READMEs. arxiv

Scientific ressources

3. AI Evaluation Standardization & Good Practices (→ justifies Hydra)

- **"REFORMS: Consensus-based Recommendations for Machine-learning-based Science"** *Science Advances* (2024) — https://www.science.org/doi/10.1126/sciadv.adk3452 → Presents a 32-question checklist developed by consensus of 19 researchers across CS, data science, mathematics, social sciences, and biomedical sciences to address failures of validity, reproducibility, and generalizability in ML-based science. Highly citable multi-disciplinary consensus document. Science
- **"A Guide to Computational Reproducibility in Signal Processing and Machine Learning"** (2021) — https://arxiv.org/pdf/2108.12383 → Advocates that researchers should plan for computational reproducibility long before any code is written, and presents practical tools and strategies to mitigate obstacles to reproducible computational experiments. arxiv

Scientific ressources

4. Medical Imaging & AI — Domain-Specific

- **"Reproducibility of Deep Learning Algorithms Developed for Medical Imaging Analysis: A Systematic Review"** *PubMed* (2023) — https://pubmed.ncbi.nlm.nih.gov/37407841/ → Conducted a systematic review of DL papers in medical imaging to evaluate whether methodology descriptions would allow reproducibility of findings, noting that lack of method details and algorithm code undermines scientific value. Directly in your domain. PubMed
- **"Reproducibility of the Methods in Medical Imaging with Deep Learning"** MIDL (2022) — https://arxiv.org/pdf/2210.11146 → Proposes guidelines for ML research in medical imaging, recommending authors show the same care when designing repositories as when writing papers, since they collectively paint the whole picture of the conducted research. arxiv
- **"Advancing Medical Imaging Research Through Standardization"** *PubMed* (2025) — https://pubmed.ncbi.nlm.nih.gov/38985896/ → Argues that objective and transparent algorithm validation on a standardized data infrastructure will enhance reproducibility and interoperability of AI systems, driving innovation and reliability in clinical applications. PubMed
- **"Standardization of imaging methods for machine learning in neuro-oncology"** *Neuro-Oncology Advances* (2021) — https://pmc.ncbi.nlm.nih.gov/pmc/articles/PMC7829470/ → Discusses how lack of reproducibility when models are applied across different institutions is a major limitation, and argues standardization of methodology and reporting is essential. nih
- **"Evaluation metrics in medical imaging AI: fundamentals, pitfalls, misapplications, and recommendations"** *European Journal of Radiology AI* (2025) — https://www.ejrai.com/article/S3050-5771(25)00028-3/fulltext → Recommends calculating metrics on blinded data, using standardized open-source tools, and comparing results on standardized benchmark datasets to enhance transparency and reproducibility. Ejrai
- **"MHub.ai: A Simple, Standardized, and Reproducible Platform for AI Models in Medical Imaging"** (2025) — https://arxiv.org/pdf/2601.10154 → Introduces a container-based platform that standardizes access to AI models, noting that key barriers include inconsistent model implementations, complex installation procedures, unclear configuration parameters, and arbitrary output formats — exactly the problems Docker + Hydra solve. arxiv
- **"Recommendations for the creation of benchmark datasets for reproducible AI in radiology"** *PMC* (2024) — https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11473745/ → Highlights that factors limiting bias in AI evaluation include data sources, anonymization steps, data format, and annotation methods, and notes existing initiatives to standardize infrastructure for validating AI software in imaging. nih

Scientific ressources

5. Best Practices Checklists & Community Standards

- **"Best Practices and Scoring System on Reviewing A.I. based Medical Imaging Papers"** SIIM / arXiv (2022) — https://arxiv.org/pdf/2202.01863 → Written from the ML practitioner's standpoint, addresses best practices for AI-based studies including dataset curation, data preprocessing, model architecture and training — specifically for medical image classification. arxiv